

MAT3110

Ana María Jiménez Güiza

September 2024

1 Compulsory assignment 1

1.1 Code previously needed for the factorizations

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import qr

# The common data set parameters
n = 30
start = -2
stop = 2
x = np.linspace(start, stop, n)
eps = 1
np.random.seed(1)
r = np.random.rand(n) * eps

# The first data set
y_1 = x * (np.cos(r+0.5*x**3)+np.sin(0.5*x**3))

# The second data set
y_2 = 4*x**5 - 5*x**4 - 20*x**3 + 10*x**2 + 40*x + 10 + r

# Choose m = 3 or m = 8
m = 3

# Create the Vandermonde matrix
A = np.vander(x, m, increasing=True)
```

1.2 QR factorization

```
""" Exercise 1: Use the QR factorization """
# QR decomposition
Q, R = qr(A)
```

```

R_1 = R[0:m, 0:m]
Q_1_transpose = Q[0:n, 0:m].T

set_1 = Q_1_transpose @ y_1
set_2 = Q_1_transpose @ y_2

# Back substitution
def back_substitution(R, y):
    n = np.shape(R)[0]
    x = np.zeros(n)

    for i in range(n-1, -1, -1):
        x[i] = (y[i] - np.dot(R[i, i+1:], x[i+1:])) / R[i, i]

    return x

solution_1_1 = back_substitution(R_1, set_1)
solution_1_2 = back_substitution(R_1, set_2)

```

1.3 Chelosky factorization

```

""" Exercise 2: Solve the normal equation using Cholesky """
B = A.T@A

set_1 = A.T @ y_1
set_2 = A.T @ y_2

def cholesky_decomposition(B):
    # B is a symmetric positive definite matrix (3x3 in this case)
    n = B.shape[0]
    R = np.zeros_like(B)
    for i in range(n):
        for j in range(i, n):
            if i == j:
                temp = sum(R[k, i] ** 2 for k in range(i-1))
                R[i, i] = np.sqrt(B[i, i] - temp)
            else:
                temp = sum(R[k, i] * R[k, j] for k in range(i-1))
                R[i, j] = (B[i, j] - temp) / R[i, i]

    return R

# Forward substitution
def forward_substitution(B, b):

```

```

n = np.shape(B)[0]
x = np.zeros(n)

# Loop over each row starting from the top (i.e., from 0 to n-1)
for i in range(n):
    x[i] = (b[i] - np.dot(B[i, :i], x[:i])) / B[i, i]

return x

R = cholesky_decomposition(B)

data_1 = forward_substitution(R.T, set_1)
data_2 = forward_substitution(R.T, set_2)

solution_2_1 = back_substitution(R, data_1)
solution_2_2 = back_substitution(R, data_2)

```

1.4 Plotting of the solutions

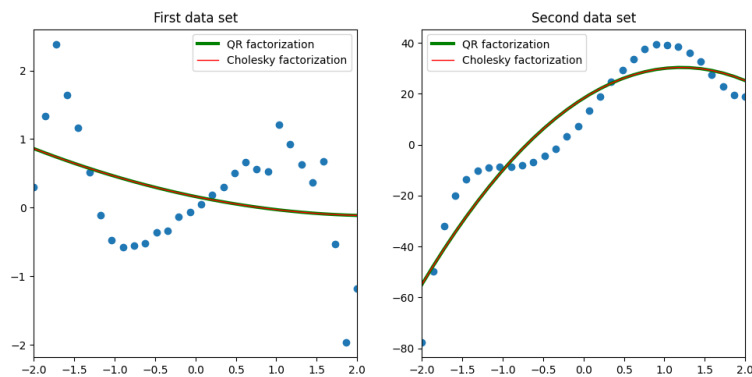


Figure 1: Plotting when $m = 3$

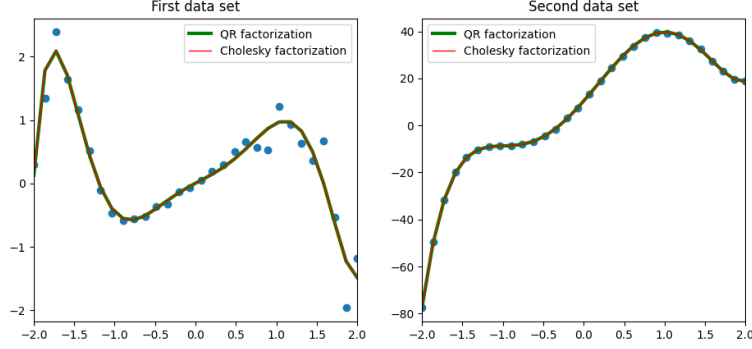


Figure 2: Plotting when $m = 8$

1.5 Differences between the two methods

The QR factorization is known for being numerically stable, particularly when dealing with poorly conditioned systems. Since Q is an orthogonal matrix, this method does not amplify errors during computation. On the other side, this method has a computational cost of $\mathcal{O}(nm^2)$

The normal equations using Cholesky decomposition is less stable than QR factorization, especially for large m . The matrix $A^T A$ can be poorly conditioned and therefore greatly amplify numerical errors, leading to inaccurate results. This also makes it more sensitive to errors, which can be problematic for illconditioned matrices. On the other side, this method has a computational cost of $\mathcal{O}(m^3)$, making it faster than QR factorization when $n > m$.

In conclusion, if m is large or the matrix A is poorly conditioned, QR factorization is generally better since it is more stable and less sensitive to conditioning issues. The normal equations with Cholesky decomposition can be faster for smaller, well-conditioned problems, but they are more prone to numerical errors.